Scope Backend RSA – Caesar_Cipher

Oct 6, 2026 · @Someone

Đợt này chỉ làm Backend cho RSA: sinh khóa, mã hóa và giải mã cả số lẫn văn bản (có chia khối), trả kèm các bước tính qua API. Không làm giao diện.

Phạm vi

Chỉ làm module tính toán và API. Mọi kết quả trả về dạng JSON, kiểm tra bằng unit test hoặc Postman.

- Sinh khóa từ p, q, e nhập vào, hoặc sinh ngẫu nhiên theo số bit.
- Mã hóa và giải mã số và văn bản. Văn bản nhận qua trường `data` hoặc file .txt.
- Chia khối theo ký tự hoặc theo byte UTF-8.
- Bắt lỗi với mã lỗi và câu thông báo tiếng Việt.
- Trả kèm các bước tính (bảng Euclid mở rộng, bảng bình phương và nhân) để sau này giao diện dùng lại.

Ngoài phạm vi: toàn bộ giao diện, padding OAEP, chữ ký số, đăng nhập, lưu lịch sử vào database, khóa thật cỡ 2048 bit. Đây là textbook RSA để học, không dùng cho dữ liệu thật.

Thứ tự triển khai

Làm theo 6 bước, bước sau dùng kết quả bước trước. File rsa-giai-thich.html dùng làm tài liệu tham chiếu và để kiểm tra chéo kết quả.

1. **Module số học** (BE-01). Viết gcd, Euclid mở rộng, nghịch đảo modulo, lũy thừa modulo nhanh, kiểm tra nguyên tố, tất cả dùng số nguyên lớn.
2. **Sinh khóa** (BE-02, BE-03). Nhập p, q, e hoặc số bit → ra n, φ(n), d và bảng Euclid. Tham chiếu phần 2 của file HTML.
3. **Văn bản và chia khối** (BE-04). Hai chế độ: theo ký tự và theo khối byte UTF-8. Tham chiếu phần 4.
4. **API mã hóa và giải mã** (BE-05). $C = P^e \bmod n$, $P = C^d \bmod n$ cho từng khối, trả kèm bảng từng bit. Tham chiếu phần 3.
5. **Bắt lỗi** (BE-06). Gắn mã lỗi cho mọi trường hợp đặc biệt.
6. **Kiểm thử** (BE-07). Chạy toàn bộ bảng test case.

Chia khối theo byte (bước 3). Đổi văn bản sang byte UTF-8. Chọn k là số byte lớn nhất sao cho $256^k \leq n - 1$, như vậy mọi khối đều nhỏ hơn n. Ghép mỗi k byte thành một số, khối cuối thiếu thì đệm byte 0x00. Giải mã xong thì tách lại thành k byte và bỏ các byte 0x00 ở

cuối. Nếu n ⩽ 256 thì k = 0, báo lỗi n quá nhỏ.

Backend

Backend giữ toàn bộ phép tính và trả về cả kết quả lẫn các bước tính để phía gọi API tự hiển thị.

ID	Task	Chi tiết	Xong khi	Phụ thuộc
BE-01	Module số học	gcd, egcd (trả cả các dòng q, r, t), modInverse, modPow (trả các dòng theo bit), isPrime: chia thử với số ⩽ 10^{12}, Miller–Rabin với số lớn hơn. Dùng BigInt	Qua test TC-01 → TC-04	–
BE-02	Sinh khóa từ p, q, e	Kiểm tra p, q nguyên tố và khác nhau, 1 < e < φ(n), gcd(e, φ(n)) = 1. Trả n, φ(n), d, bảng Euclid. Khi gcd ≠ 1 thì gợi ý e hợp lệ nhỏ nhất	Qua TC-01, TC-06 → TC-09	BE-01
BE-03	Sinh khóa ngẫu nhiên	Nhận số bit của n (16, 32, 64, 128). Sinh p, q ngẫu nhiên bằng Miller–Rabin, ưu tiên e = 65537, nếu không hợp lệ thì chọn e khác	n đúng số bit, mã hóa rồi giải mã ra đúng	BE-01
BE-04	Mã hóa văn bản thành khối	Chế độ "char": mỗi ký tự Unicode là một khối. Chế độ "block": byte UTF-8, k byte mỗi khối với 256^k ⩽ n − 1, đệm 0x00. Có hàm ngược lại	Qua TC-10 → TC-13	BE-01
BE-05	API mã hóa và giải mã	Nhận khóa + dữ liệu (số hoặc văn bản), trả kết quả, danh sách khối và bảng modPow của từng khối	Qua TC-02, TC-05	BE-02, BE-04
BE-06	Mã lỗi và validation	Mọi lỗi trả HTTP 400 với mã lỗi theo bảng Bắt lỗi. Giới hạn đầu vào: văn bản ⩽ 10.000 ký tự, file .txt ⩽ 1 MB	Mỗi dòng bảng Bắt lỗi có 1 test	BE-02, BE-05

ID	Task	Chi tiết	Xong khi	Phụ thuộc
BE-07	Unit test	Viết test cho toàn bộ bảng test case	Tất cả TC-01 → TC-13 qua	BE-01 → BE-06

API

Mọi số gửi và nhận dưới dạng chuỗi, vì số lớn trong JSON sẽ bị làm tròn khi vượt 2^{53}.

Endpoint	Request	Response
POST /api/rsa/keys	`{p, q, e}`	`{n, phi, e, d, publicKey, privateKey, egcdSteps[]}`
POST /api/rsa/keys/random	`{bits}`	giống /keys, thêm `{p, q}`
POST /api/rsa/encrypt	`{e, n, inputType: "number" | "text", mode: "char" | "block", data}`	`{blocks[], cipher[], blockSize, steps[]}`
POST /api/rsa/decrypt	`{d, n, inputType, mode, cipher[]}`	`{blocks[], plaintext, steps[]}`

Văn bản có thể gửi băng trường `data` hoặc upload file .txt qua multipart/form-data (trường `file`).

Lỗi trả HTTP 400 với `{code, message, field}`.

Bắt lỗi

Mã lỗi	Trường hợp	Thông báo
NOT_INTEGER	p, q, e, P hoặc bản mã không phải số nguyên không âm	"Giá trị phải là số nguyên không âm."
NOT_PRIME	p hoặc q không phải số nguyên tố	"p = 15 không phải số nguyên tố."
SAME_PRIME	p = q	"p và q phải khác nhau."
E_OUT_OF_RANGE	e ⩽ 1 hoặc e ⩾ φ(n)	"e phải thỏa 1 < e < φ(n) = 160."

Mã lỗi	Trường hợp	Thông báo
E_NOT_COPRIME	gcd(e, φ(n)) ≠ 1	"gcd(10, 160) = 10, không tồn tại d. Gợi ý e = 3."
PRIME_TOO_LARGE	p hoặc q > 10^{12} khi nhập tay	"Hãy dùng p, q ⩽ 10^{12} hoặc sinh khóa ngẫu nhiên."
P_TOO_LARGE	Số P ⩾ n (chế độ số) hoặc mã ký tự ⩾ n (chế độ char)	"P = 200 ⩾ n = 187. Hãy chia khối hoặc dùng n lớn hơn."
N_TOO_SMALL	Chế độ block mà n ⩽ 256	"n phải lớn hơn 256 để chứa ít nhất 1 byte mỗi khối."
CIPHER_TOO_LARGE	Một số trong bản mã ⩾ n	"Bản mã không hợp lệ với khóa này."
DECODE_FAILED	Giải mã ra byte không phải UTF-8 hợp lệ (thường do sai khóa)	"Không đọc được văn bản, kiểm tra lại khóa riêng."
EMPTY_INPUT	Trường data hoặc file rỗng	"Dữ liệu đầu vào đang rỗng."
FILE_INVALID	File không phải .txt hoặc > 1 MB	"Chỉ nhận file .txt tối đa 1 MB."

Test case

Các giá trị dưới đây đã được tính và chạy kiểm tra. TC-01, TC-02 là ví dụ trong giáo trình Stallings, khớp với slide.

ID	Đầu vào	Kết quả mong đợi
TC-01	Sinh khóa p = 17, q = 11, e = 7	n = 187, φ(n) = 160, d = 23
TC-02	KU = {7, 187}, P = 88	C = 11; giải mã 11 bằng d = 23 ra 88
TC-03	Sinh khóa p = 61, q = 53, e = 17	n = 3233, φ(n) = 3120, d = 2753
TC-04	KU = {17, 3233}, P = 65	C = 2790; giải mã ra 65
TC-05	KU = {7, 187}, P = 200	Lỗi P_TOO_LARGE
TC-06	p = 15, q = 11, e = 7	Lỗi NOT_PRIME

ID	Đầu vào	Kết quả mong đợi
TC-07	p = 11, q = 11, e = 7	Lỗi SAME_PRIME
TC-08	p = 17, q = 11, e = 10	Lỗi E_NOT_COPRIME, gợi ý e = 3
TC-09	p = 17, q = 11, e = 200	Lỗi E_OUT_OF_RANGE
TC-10	Chế độ char, n = 187, văn bản "Việt"	Lỗi P_TOO_LARGE ("ệ" có mã 7879)
TC-11	Chế độ char, p = 101, q = 113, e = 3533, "Xin chào Việt Nam"	Bản mã: 1191 4861 8266 5410 9661 4909 11155 6070 5410 8077 4861 6970 7780 5410 4334 290 10050; giải mã ra đúng câu gốc
TC-12	Chế độ block, p = 257, q = 263, e = 3, "Hi!"	n = 67591, d = 44715, k = 2; khối 18537, 8448; bản mã 37222 6468; giải mã ra "Hi!"
TC-13	Chế độ block, n = 187	Lỗi N_TOO_SMALL

Câu hỏi mở

- [ ] RSA là tuần mấy và deadline khi nào?
- [ ] Backend tuần 1 viết bằng gì? Python có sẵn số nguyên lớn, JavaScript cần dùng BigInt.
- [ ] Sinh khóa ngẫu nhiên có cần đến 512 bit trở lên không, hay chỉ tối đa 128 bit để demo và xem bảng từng bước?
- [ ] Bản mã xuất dạng số thập phân (như hiện tại) hay thêm dạng hex / base64?

Nguồn

- The RSA Algorithm – Đại học Mosul (ví dụ 17, 11, 7, 88)
- Wycheproof – RSA notes (kích thước khóa, e = 65537)
- RFC 8017 – PKCS #1 v2.2 (OAEP)